Differentiable solvers for time series segmentation

Time series segmentation (change-point detection, piecewise approximation, regime discovery) relies on discrete optimization that is non-differentiable, blocking its use as a layer in trainable pipelines. Smoothing a fixed-grid dynamic program is well understood; the real obstacle is *model selection*, since the optimizer also chooses how many segments to use. This internship asks whether that segment-count decision can itself be made differentiable, end to end.

§1. Organization

- **Start date**: As soon as possible (flexible).
- **Localization**: IRISA, Rennes.
- **Supervision**:
  - Romain Tavenard (Prof., Université Rennes 2, MALT Inria team, IRISA, Rennes).
  - Paul Boniol (Researcher, VALD Inria team, ENS Paris).

§2. Context

Time series segmentation is a family of related problems that partition a signal into homogeneous or meaningful pieces. It includes change-point detection / optimal partitioning (locating shifts in the statistical properties of a signal), piecewise approximation (representing a signal as a sequence of constant or linear segments), and regime- or motif-based segmentation (Truong et al., 2020). The workhorse formulation is *penalized optimal partitioning*: one minimizes, over all partitions, a data-fit cost plus a penalty λ on the number of breakpoints. This is a dynamic program, solved exactly and in linear time by PELT (Killick et al., 2012), and the penalty λ is what sets how many segments are returned. Like all such solvers, it is *non-differentiable*.

Non-differentiability prevents these solvers from being used as layers inside end-to-end trainable pipelines: one cannot learn a representation or cost function whose induced segmentation is better for a downstream task, nor jointly train a forecasting or anomaly-detection model that exploits inferred segment structure. Two ingredients exist that this internship will build on. First, generic tools make combinatorial solvers differentiable: *continuous relaxation*, which smooths the dynamic-programming recursion by replacing its min with a soft counterpart (Mensch & Blondel, 2018), and *stochastic perturbation*, which randomizes the solver's inputs and averages to obtain gradients while treating the solver as a black box (Berthet et al., 2020). Second, the change-point community has a long line of *penalty learning*: since λ governs the number of segments, it can be fit from labelled data by interval regression (Hocking et al., 2013) or predicted by a neural network (Truong & Runge, 2025); a differentiable change-point detector has also been proposed for one specific paradigm (Koley et al., 2023).

§3. Scientific challenge

The two bodies of work above each leave the central difficulty untouched, from opposite sides. Differentiable dynamic programming smooths the recursion only on a *fixed* trellis (as in dynamic time warping): it never decides how many segments to use. Penalty learning *does* decide the number of segments, but the penalty is fit by a predictor that sits *outside* the partitioning solver, which remains a fixed, non-differentiable oracle, so gradients never flow through the segmentation itself. What is missing is exactly the bridge: a *model-selection step that is differentiable through the solver*. The obstacle is intrinsic: the number of segments is an integer chosen by trading data fit against a breakpoint penalty, so the solution is piecewise constant and the count jumps discontinuously as the signal, cost function, or penalty vary. The generic relaxation and

perturbation tools are designed for exactly this kind of discontinuity, but have not been brought to bear on the segment-count choice. The open question is therefore: **can the model-selection step be made differentiable through the partitioning solver, so that a representation, a cost function, *and* the number of segments are learned jointly and end-to-end?** Two routes are natural, and each adapts a known tool to the penalized partitioning problem: smoothing the penalized DP recursion so the segment count becomes a soft, differentiable quantity, or perturbing the penalized solver and averaging so its output varies smoothly with the costs and the penalty. A secondary question is which of the two yields more stable segments and gradients, and whether the answer differs across segmentation paradigms.

§4. Goals

Concretely, the candidate will:

1. formalize penalized optimal partitioning so that the segment costs and the penalty λ are explicit, differentiable inputs — the object both methods below act on;
2. develop the *relaxation* route: smooth the penalized DP recursion so that the number of segments becomes a soft, differentiable quantity, and derive gradients with respect to the costs and λ;
3. develop the *perturbation* route: treat the penalized solver as a black box, perturb its inputs and average, and obtain gradients through the resulting smooth expected segmentation;
4. benchmark both routes against the exact discrete solver and learned baselines using a unified segmentation toolkit (Chavelli et al., 2026), and demonstrate one end-to-end use case jointly learning a feature extractor and λ (e.g. on regime-switching series or an anomaly-detection benchmark).

Candidates should possess:

- good Python coding skills,
- knowledge of dynamic programming and time-series fundamentals,
- broader familiarity with time-series representations is a plus,
- ability to communicate in English in a scientific context.

Funding for a Ph.D. has already been secured: upon a successful internship, the candidate will be offered the opportunity to continue this work as a fully funded doctoral thesis. A successful candidate will develop or reinforce skills key to that continuation, namely:

- a practical and theoretical understanding of differentiable combinatorial optimization,
- knowledge of how segmentation paradigms map to dynamic programs,
- experience designing benchmarks and end-to-end training pipelines for time series.

§5. Bibliography

Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., & Bach, F. (2020). *Learning with Differentiable Perturbed Optimizers.* NeurIPS.

Chavelli, F., Ermshaus, A., Yang, F., Boniol, P., Schäfer, P., & Paparrizos, J. (2026). *tsseg: A Python Library for Time Series Segmentation.* ECML-PKDD.

Hocking, T. D., Rigaill, G., Vert, J.-P., & Bach, F. (2013). *Learning Sparse Penalties for Change-Point Detection using Max Margin Interval Regression.* ICML.

Killick, R., Fearnhead, P., & Eckley, I. A. (2012). *Optimal Detection of Changepoints with a Linear Computational Cost.* Journal of the American Statistical Association.

Koley, P., De, A., Ganguly, N., et al. (2023). *Differentiable Change-point Detection with Temporal Point Processes.* AISTATS.

Mensch, A., & Blondel, M. (2018). *Differentiable Dynamic Programming for Structured Prediction and Attention.* ICML.

Truong, C., Oudre, L., & Vayatis, N. (2020). *Selective Review of Offline Change Point Detection Methods.* Signal Processing.

Truong, C., & Runge, V. (2025). *Learning Penalty for Optimal Partitioning via Automatic Feature Extraction.* arXiv:2505.07413.